

DISCOVERYAI PROJECT REPORT

AI-Powered Digital Product Discovery Platform

Sowjanya Kamtam-000798156

Harshita Guduru – 000797805

Darshan Joshi – 000797697

Abstract

DiscoveryAI is an AI-based platform designed to automate the early stages of software development, including requirement gathering, analysis, design modeling, and partial code generation. Instead of manually writing requirements and documentation, users describe their idea in natural language, and DiscoveryAI uses Natural Language Processing (NLP) and AI models to transform that input into structured Software Requirements Specification (SRS) content, design artifacts, and sample backend/frontend code.

This report is organized into three phases:

- **Phase 1 – Analysis:** Problem identification and requirement gathering
- **Phase 2 – Design:** System architecture, data model, class design, and UI/UX
- **Phase 3 – Program Synthesis:** Code generation and evaluation

PHASE 1 – ANALYSIS

1.1 Problem Identification

In most software projects, the **discovery phase** (where requirements are discussed and documented) is slow, manual, and error-prone. Teams often:

- Struggle to write clear and complete requirements
- Spend significant time creating formal SRS documents
- Miscommunicate between technical and non-technical stakeholders
- Rework features due to vague or misunderstood requirements

DiscoveryAI aims to solve these issues by automating parts of this process. The system:

- Accepts free-text descriptions of software ideas
- Uses NLP to extract requirements

- Organizes them into structured formats
 - Reduces manual effort in early project phases
-

1.2 Project Goals

- Automate requirement extraction from natural language.
- Support non-technical users in describing software ideas.
- Generate structured SRS sections automatically.
- Provide a foundation for later design and implementation.

1.3 Stakeholders and Needs

Stakeholder	Need
Product Owner	Clear, organized project requirements
Developer	Well-structured SRS and use case information
UX Designer	User flows and hints for interface structure
Student/Learner	Example of AI-assisted software engineering tool
AI System	Clean, labeled training and inference data

1.4 Functional Requirements (FR)

- FR1: The system shall allow users to input free-text project descriptions.
- FR2: The system shall extract functional requirements using NLP.
- FR3: The system shall classify requirements by type (functional/non-functional).
- FR4: The system shall generate summaries, use case descriptions, and user stories.
- FR5: The system shall export generated requirements and documentation.

1.5 Non-Functional Requirements (NFR)

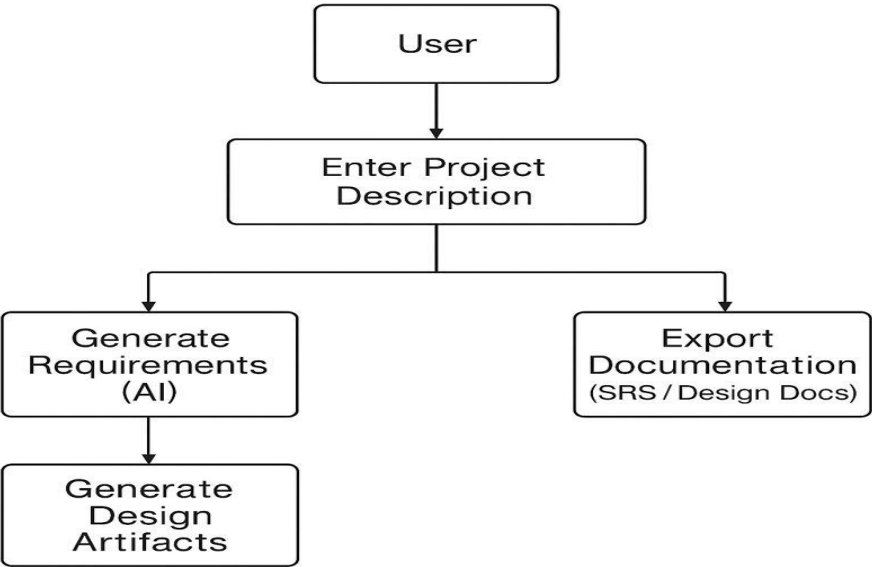
ID	Non-Functional Requirement
NFR1	The system shall respond within 3 seconds for average input size.
NFR2	The user interface shall be simple and intuitive.
NFR3	All user data shall be transmitted securely.

NFR4	The system shall achieve at least 99% uptime in production.
------	---

1.6 Use Case Summary

Use Case ID	Name	Primary Actor
UC-01	Enter project description	User
UC-02	Generate requirements	System (AI)
UC-03	Generate diagrams/flows	System
UC-04	Export documentation	User

Use Case Diagram:



PHASE 2 – DESIGN:

2.1 System Architecture Overview

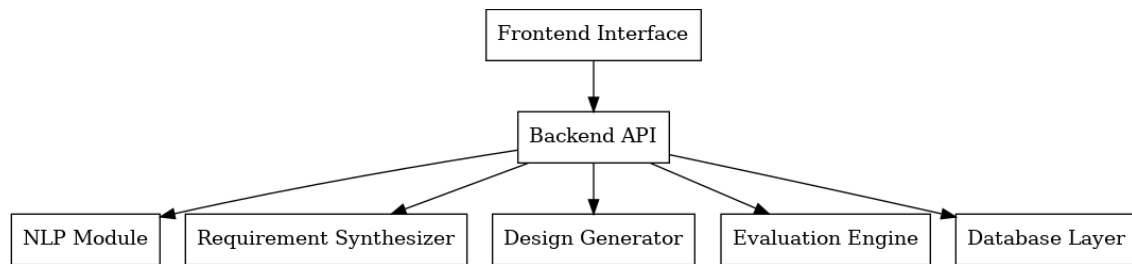
DiscoveryAI follows a **modular, microservices-inspired architecture**. Even if it is deployed as a single application in early stages, components are logically separated so they can scale independently in the future.

2.1.1 Architecture Layers

1. **Frontend Web Interface**
2. **Backend API Layer**

3. AI Processing Modules

4. Database Layer



The system uses Microservices with an AI-Oriented Modular Architecture, consisting of:

1. Frontend Interface:

- Developed using a modern JavaScript framework (e.g., React).
- Collects inputs like project goals, descriptions, and constraints.
- Displays AI-generated outputs such as requirements and diagrams.
- Implements a **wizard-style interface** guiding the user step by step.

2.Backend API (REST or GraphQL)

- Manages user sessions, handles queries, triggers AI modules.
- Implements REST (or GraphQL) endpoints for communication.
- Receives JSON payloads from the frontend.
- Validates and forwards them to AI modules.
- Returns structured responses (arrays of requirements, objects, etc.).

Typical endpoints:

- POST /generate-requirements
- POST /generate-usecases
- GET /projects/id}
- POST /export

3.AI Modules

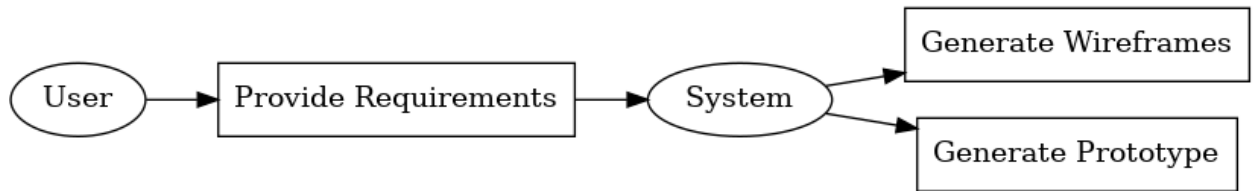
- NLP (Natural Language Processing Engine): Parses user answers.
- Requirement Synthesizer: Converts text into requirements & goals.
- Design Generator: Produces wireframes, flows, low-fi prototypes.
- Evaluation Engine: Ensures quality & consistency.

4.Database Layer

- Stores projects, user inputs, generated artifacts.

Detailed Design Artifacts:

1. Use Case Diagram:



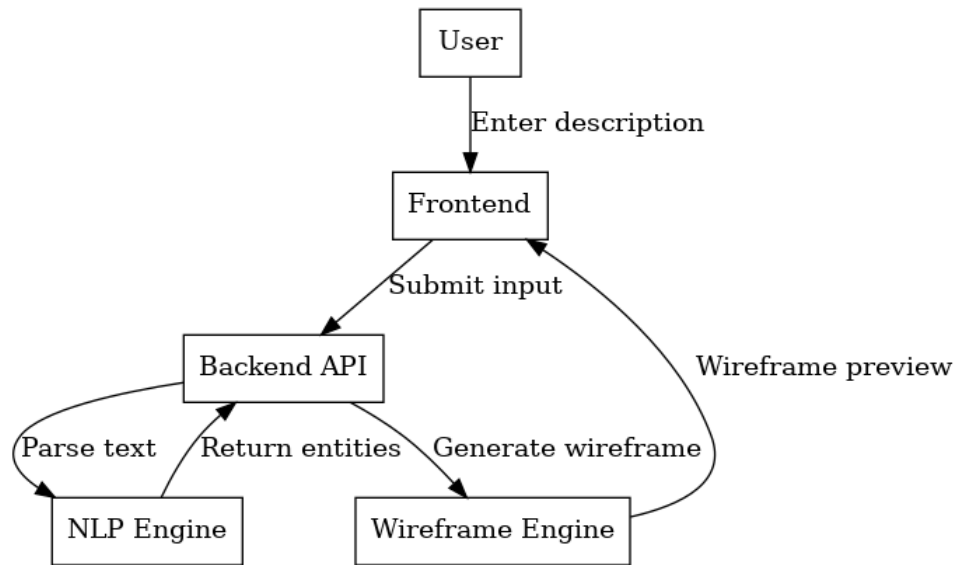
Actors:

- User
- System AI Engine
- Prototype Generator

Use Cases:

- Provide Requirements
- View Generated Requirements
- Generate User Flows
- Generate Wireframes
- GeneratePrototype
- Export Artifacts

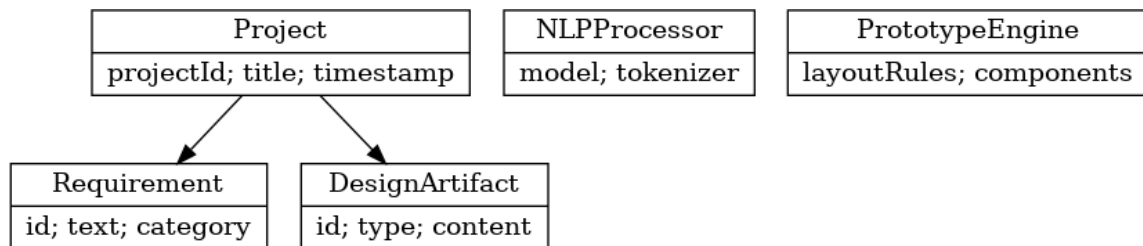
2. Sequence Diagram:



Sequence Diagram illustrates the step-by-step communication between the User, Frontend, Backend, NLP Engine, and Wireframe Generator during requirement submission and wireframe generation.

- Shows the temporal order of operations
- Demonstrates request-response interactions
- Visualizes internal communication flows
- Confirms system timing and flow correctness

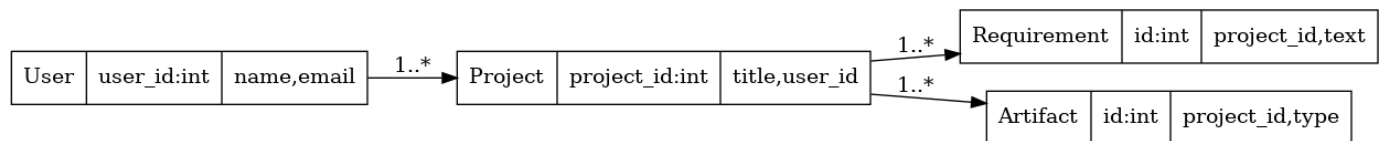
3. Class Diagram:



Core Classes:

Class	Attributes	Methods
Project	projectId, title, timestamp	save (), load ()
Requirement	id, text, category	classify (), update ()
Class	Attributes	Methods
DesignArtifact	id, type, content	generate (), export ()
NLPProcessor	model, tokenizer	parse (), extractEntities()
PrototypeEngine	layoutRules, components	generatePrototype()

4. Data Model:



Entity-Relationship Diagram illustrating how users, projects, requirements, and design artifacts relate within the system's database.

Entities:

- User
- Project
- Requirement
- Artifact
- WorkflowStep

Relationships:

- One User → many Projects
- One Project → many Requirements
- One Project → many Artifacts

- One Artifact → many WorkflowSteps

5. UI/UX Design Overview

DiscoveryAI's user interface is designed as a **wizard-style** flow.

2.4.1 Main Characteristics

Step 1: User enters a high-level description of the project.

Step 2: System displays extracted requirements in real-time.

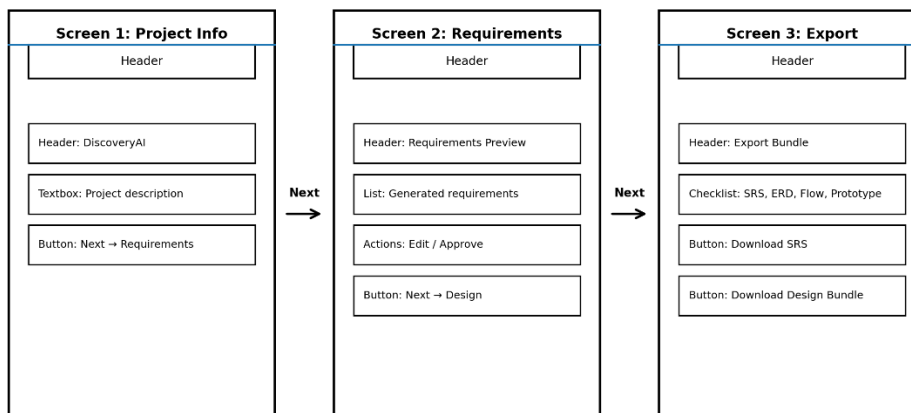
Step 3: System suggests diagrams or logical groupings.

Step 4: User can refine, accept, or edit the generated content.

Step 5: User exports a document (SRS, PDF, DOCX).

6. User Flow Diagram:

UI / UX Wireframe - DiscoveryAI Wizard (Modern Minimal B/W)



PHASE 3 – PROGRAM SYNTHESIS:

3.1 Goal

In this phase, DiscoveryAI uses structured requirements to generate partial backend and frontend code. The aim is not full automation, but to provide boilerplate code that accelerates implementation.

Layer	Technology
Backend	FastAPI (Python)
Frontend	React.js
NLP	OpenAI GPT

Backend code:

```
backend > main.py > RequirementInput
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from typing import List
4 from fastapi.middleware.cors import CORSMiddleware
5
6 app = FastAPI()
7
8 app.add_middleware(
9     CORSMiddleware,
10     allow_origins=["http://localhost:5173"],
11     allow_credentials=True,
12     allow_methods=["*"],
13     allow_headers=["*"],
14 )
15
16 class RequirementInput(BaseModel):
17     text: str
18
19 class RequirementOutput(BaseModel):
20     requirements: List[str]
21
22 @app.post("/generate-requirements", response_model=RequirementOutput)
23 async def generate_requirements(input_data: RequirementInput):
24     return RequirementOutput(requirements=[
25         "The system shall allow users to sign up.",
26         "The system shall allow users to log in.",
27         "The system shall generate requirements using AI.",
28         "The system shall export SRS and design documents."
29     ])
30 @app.get("/")
31 def home():
32     return {"message": "Backend running. Go to /docs"}
33
```

Output:

Code	Details
200	Response body <pre>{ "requirements": ["The system shall allow users to sign up.", "The system shall allow users to log in.", "The system shall generate requirements using AI.", "The system shall export SRS and design documents."] }</pre>

Frontend Code:

```
frontend > src > App.jsx > App > generateReqs
1 import { useState } from "react";
2
3 export default function App() {
4     const [text, setText] = useState("");
5     const [reqs, setReqs] = useState([]);
6
7     const generateReqs = async () => {
8         const res = await fetch("http://127.0.0.1:8000/generate-requirements", {
9             method: "POST",
10            headers: { "Content-Type": "application/json" },
11            body: JSON.stringify({ text }),
12        });
13
14        const data = await res.json();
15        setReqs(data.requirements);
16    };
17
18    return (
19        <div style={{ maxWidth: 800, margin: "40px auto", fontFamily: "Arial" }}>
20            <h2>DiscoveryAI - Requirement Generator</h2>
21        </div>
22    );
23 }
```

```

22     <textarea
23       rows="5"
24       style={{ width: "100%", padding: 10 }}
25       placeholder="Describe your app idea..."
26       value={text}
27       onChange={(e) => setText(e.target.value)}
28     />
29
30     <button
31       onClick={generateReqs}
32       style={{ marginTop: 10, padding: "10px 16px" }}
33     >
34       Generate Requirements
35     </button>
36
37     <h3 style={{ marginTop: 20 }}>Generated Requirements:</h3>
38     <ul>
39       {reqs.map((r, i) => (
40         <li key={i}>{r}</li>
41       ))}
42     </ul>
43   </div>
44 );
45 }
46

```

Output:

DiscoveryAI - Requirement Generator

I want an app for students to submit assignments

Generate Requirements

Generated Requirements:

- The system shall allow users to sign up.
- The system shall allow users to log in.
- The system shall generate requirements using AI.
- The system shall export SRS and design documents.

Evaluation

- **Strengths:**
 - Reduces time spent on boilerplate code.
 - Produces consistent requirement wording.
 - Helps inexperienced users create proper documentation.
- **Limitations:**
 - Generated code still needs human review.
 - Edge cases, security, and validations must be manually added.

Conclusion

DiscoveryAI demonstrates that AI can greatly support the discovery phase of software projects. By automating requirement extraction, design documentation, and initial code skeletons, the system saves time, improves clarity, and supports students and professionals in software engineering tasks. The three phases—Analysis, Design, and Program Synthesis—show a complete workflow from idea to partially implemented solution.